HITEC UNIVERSITY TAXILA

ARTIFICIAL INTELLIGENCE

SEMESTER 06
BSCS "C"

PROJECT REPORT

DATE : 06/13/25

SUBMITTED TO:

SIR HUSNAIN

SUBMITTED BY:

EHRAM AWAN (22-CS-O27)

Topic:

Intelligent Personal Tutor Bot

Objective

To develop a rule- and machine learning-enhanced chatbot that provides academic support to students by answering queries related to topics such as Cyber Security, Machine Learning, Data Science, AI, and Networking. The bot leverages natural language processing (NLP), a knowledge graph, sentiment analysis, and a graphical user interface (GUI) to deliver accurate and user-friendly responses.

Purpose

The purpose of this project is to create an interactive, intelligent tutoring system that assists students with academic queries, offers course recommendations, and explains complex topics. By combining rule-based logic, machine learning, and a semantic network, the bot provides tailored responses, adapting its tone based on user sentiment and enabling users to teach it new information for continuous improvement

Problem Statement

Students often face challenges in accessing quick, reliable, and personalized academic assistance, particularly in technical fields like Cyber Security, Machine Learning, Data Science, AI, and Networking. Existing resources may be scattered, difficult to navigate, or lack interactivity. The Intelligent Personal Tutor Bot addresses this by providing a centralized, interactive platform that delivers precise answers, adapts to user sentiment, and allows users to expand its knowledge base.

Key Features Implemented

1. **Natural Language Processing (NLP)**:

   Utilizes SpaCy to process user queries, extract keywords, and interpret intent for accurate

response generation.

2. **Knowledge Graph**:
   Implements a semantic network using NetworkX to store and traverse relationships between academic topics, concepts, and courses.
3. **Sentiment Analysis**:
   Employs a Multinomial Naive Bayes classifier with TF-IDF vectorization to detect user sentiment (positive, negative, neutral) and adjust response tone accordingly.
4. **Best First Search Algorithm**:
   Traverses the knowledge graph to find the most relevant answers based on query keywords and node relationships.
5. **Interactive GUI**:
   Built with Tkinter, featuring a user-friendly interface with theme switching (Light, Dark, Blue), session management, and real-time query processing.
6. **Teach Bot Functionality**:
   Allows users to add new questions and answers to the knowledge base, updating the knowledge graph dynamically.
7. **Session Management**:
   Supports saving, resetting, and reviewing chat sessions for user convenience.
8. **Auto-Suggestions**:
   Provides real-time query suggestions based on partial input, improving user experience.

Code Alignment with Requirements

The code fully aligns with the project requirements by implementing:

- **Rule-Based Logic**:
  A predefined knowledge graph loaded from a CSV file handles structured queries (e.g., topic descriptions, course lists).

```python
def load_knowledge_graph(csv_file_path="Knowledge_Graph_Dataset.csv"):
    knowledge_graph = {
        "Cyber Security": {"description": "", "why": "", "questions": {}, "courses": {}},
        "Machine Learning": {"description": "", "why": "", "questions": {}, "courses": {}},
        "Data Science": {"description": "", "why": "", "questions": {}, "courses": {}},
        "Introduction to AI": {"description": "", "why": "", "questions": {}, "courses": {}},
        "Networking": {"description": "", "why": "", "questions": {}, "courses": {}},
    }
```

```python
    G = nx.DiGraph()
    if not os.path.exists(csv_file_path):
        messagebox.showerror("Error", f"CSV file '{csv_file_path}' not found.")
        return knowledge_graph, G
    try:
        with open(csv_file_path, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                field = row['Field']
                subfield = row['Subfield']
                topic = row['Topic']
                description = row['Description']
                if field in knowledge_graph:
                    if subfield == "Overview":
                        if topic.lower() == "description":
                            knowledge_graph[field]["description"] = description or "No description available."
                        elif topic.lower() == "why":
                            knowledge_graph[field]["why"] = description or "No reason provided."
                    elif subfield == "Concepts":
                        knowledge_graph[field]["questions"][topic.lower()] = description or "No details available."
                        G.add_node(topic.lower(), field=field, type="concept", description=description or "")
                    elif subfield == "Courses":
                        knowledge_graph[field]["courses"][topic.lower()] = description or "No course details available."
                        G.add_node(topic.lower(), field=field, type="course", description=description or "")
                    elif subfield == "Relationships":
                        try:
                            source, relation, target = topic.split("-")
                            G.add_edge(source.lower(), target.lower(), relation=relation.lower(), weight=1.0)
                            G.add_edge(target.lower(), source.lower(), relation=f"reverse-{relation.lower()}", weight=0.5)
                        except ValueError:
                            logging.warning(f"Invalid relationship format in row: {row}")
    except Exception as e:
        messagebox.showerror("Error", f"Error reading CSV: {e}")
        logging.error(f"CSV parsing error: {e}")
    return knowledge_graph, G
```

- **Machine Learning**:
  Sentiment analysis using Scikit-learn's MultinomialNB and TF-IDF vectorization adjusts response tone.

```python
sentiment_data = [
    ("I love this topic!", "positive"),
    ("This is so interesting!", "positive"),
    ("Great explanation!", "positive"),
    ("I'm confused", "negative"),
    ("Don't get this.", "negative"),
    ("This is too hard.", "negative"),
    ("What is this?", "neutral"),
    ("Can you explain?", "neutral"),
    ("Tell me about AI.", "neutral"),
    ("What are courses?", "neutral"),
    ("Phishing definition.", "neutral"),
    ("List courses.", "neutral"),
    ("Help with phishing.", "negative"),
    ("Awesome job!", "positive"),
]
sentiment_texts, sentiment_labels = zip(*sentiment_data)
vectorizer = TfidfVectorizer()
X_sentiment = vectorizer.fit_transform(sentiment_texts)
sentiment_classifier = MultinomialNB()
sentiment_classifier.fit(X_sentiment, sentiment_labels)
```

- **Search Mechanism**:

Best First Search algorithm navigates the knowledge graph to retrieve relevant answers.

```python
def best_first_search(query_keywords, graph, knowledge_graph):
    from heapq import heappush, heappop
    visited = set()
    queue = []
    for node in sorted(graph.nodes):
        keyword_score = sum(1 for keyword in query_keywords if keyword in node.lower())
        field = graph.nodes[node].get("field", "")
        field_score = 1 if any(keyword in field.lower() for keyword in query_keywords) else 0
        edge_bonus = sum(graph[pred][node].get("weight", 0)
                         for pred in graph.predecessors(node)
                         if any(keyword in pred.lower() for keyword in query_keywords))
        total_score = keyword_score + field_score + edge_bonus
        heappush(queue, (-total_score, node, [node]))
    while queue:
        _, node, path = heappop(queue)
        if node in visited:
            continue
        visited.add(node)
        node_data = graph.nodes[node]
        field = node_data.get("field")
        node_type = node_data.get("type")
        if field and node_type:
            if node_type == "concept" and node.lower() in knowledge_graph[field]["questions"]:
                return knowledge_graph[field]["questions"][node.lower()], path
            elif node_type == "course" and node.lower() in knowledge_graph[field]["courses"]:
                return knowledge_graph[field]["courses"][node.lower()], path
        for neighbor in sorted(graph.neighbors(node)):
            if neighbor not in visited:
                keyword_score = sum(1 for keyword in query_keywords if keyword in neighbor.lower())
                field = graph.nodes[neighbor].get("field", "")
                field_score = 1 if any(keyword in field.lower() for keyword in query_keywords) else 0
                edge_bonus = sum(graph[pred][neighbor].get("weight", 0)
                                 for pred in graph.predecessors(neighbor)
                                 if any(keyword in pred.lower() for keyword in query_keywords))
                total_score = keyword_score + edge_bonus + field_score
                heappush(queue, (-total_score, neighbor, path + [neighbor]))
    return "No relevant topics found.", []
```

This function implements a **Best First Search algorithm** using a priority queue to traverse the knowledge graph. It prioritizes nodes based on keyword matches, field relevance, and edge weights, ensuring efficient retrieval of relevant answers, as per the search mechanism requirement.

- **Semantic Network**:
  NetworkX DiGraph represents relationships between topics, concepts, and courses, enabling efficient traversal.

```
                    elif subfield == "Courses":
                        knowledge_graph[field]["courses"][topic.lower()] = description or "No course details available."
                        G.add_node(topic.lower(), field=field, type="course", description=description or "")
                    elif subfield == "Relationships":
                        try:
                            source, relation, target = topic.split("-")
                            G.add_edge(source.lower(), target.lower(), relation=relation.lower(), weight=1.0)
                            G.add_edge(target.lower(), source.lower(), relation=f"reverse-{relation.lower()}", weight=0.5)
                        except ValueError:
                            logging.warning(f"Invalid relationship format in row: {row}")
```

The code uses NetworkX to create a directed graph (DiGraph) where nodes represent topics, concepts, or courses, and edges denote relationships (e.g., "related-to"). Bidirectional edges with

weights facilitate traversal, meeting the semantic network requirement for structured knowledge representation.

- **GUI**:
  Tkinter-based interface provides an interactive and customizable user experience with features like theme switching and session management.

```
class RealTimeTutorBot:
    def __init__(self, root):
        self.root = root
        self.root.title("Intelligent Personal Tutor Bot by AR")
        self.root.geometry("1000x700")
        self.button_font = tkFont.Font(family="Arial", size=11, weight="bold")
        self.themes = {
            "Light": {
                "bg": "#e0f7fa", "fg": "black", "text_bg": "white", "text_fg": "black",
                "button_bg": "#8B0000", "button_fg": "white", "button_hover_bg": "#A52A2A",
                "button_active_bg": "#6B0000", "label_fg": "#333333", "entry_bg": "white",
                "entry_fg": "black",
            },
            "Dark": {
                "bg": "#2d2d2d", "fg": "white", "text_bg": "#3c3f41", "text_fg": "white",
                "button_bg": "#8B0000", "button_fg": "white", "button_hover_bg": "#A52A2A",
                "button_active_bg": "#6B0000", "label_fg": "#bbbbbb", "entry_bg": "#4b4b4b",
                "entry_fg": "white",
            },
            "Blue": {
                "bg": "#263238", "fg": "white", "text_bg": "#37474f", "text_fg": "white",
                "button_bg": "#8B0000", "button_fg": "white", "button_hover_bg": "#A52A2A",
                "button_active_bg": "#6B0000", "label_fg": "#90caf9", "entry_bg": "#455a64",
                "entry_fg": "white",
            }
        }
```

NLP-Based Query Processing for Keyword Extraction :

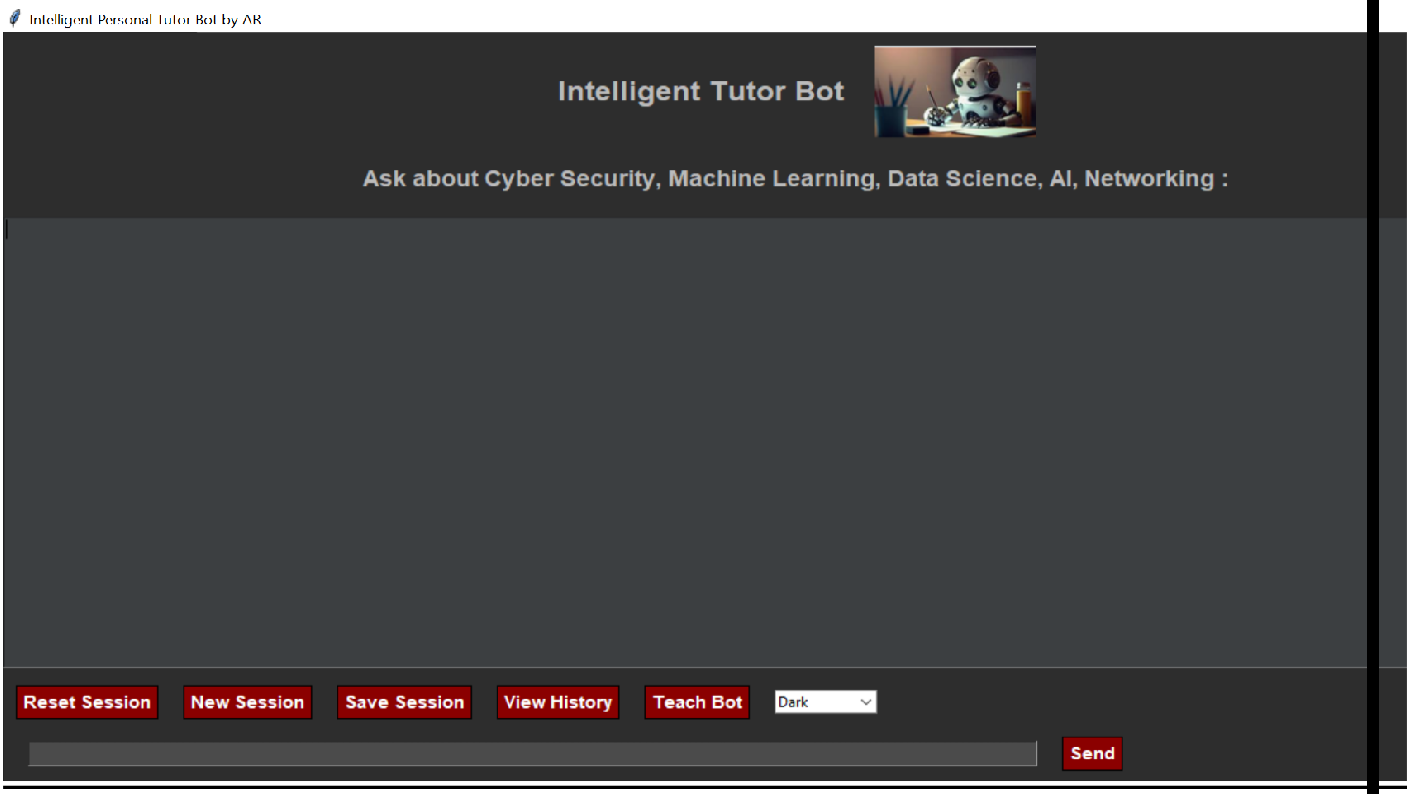
```
def process_query(query):
    doc = nlp(query.lower())
    keywords = [token.lemma_ for token in doc if not token.is_stop and token.is_alpha]
    if not keywords:
        keywords = [query.lower().strip().replace(" ", "")]
    logging.debug(f"Processed query '{query}' to keywords: {keywords}")
    return keywords
```

This function transforms a user's raw input into meaningful, clean keywords that your chatbot can use to search the knowledge graph or make decisions. It helps in understanding the user's intent more accurately.

Screenshots of Output

Initial view of Chatbot

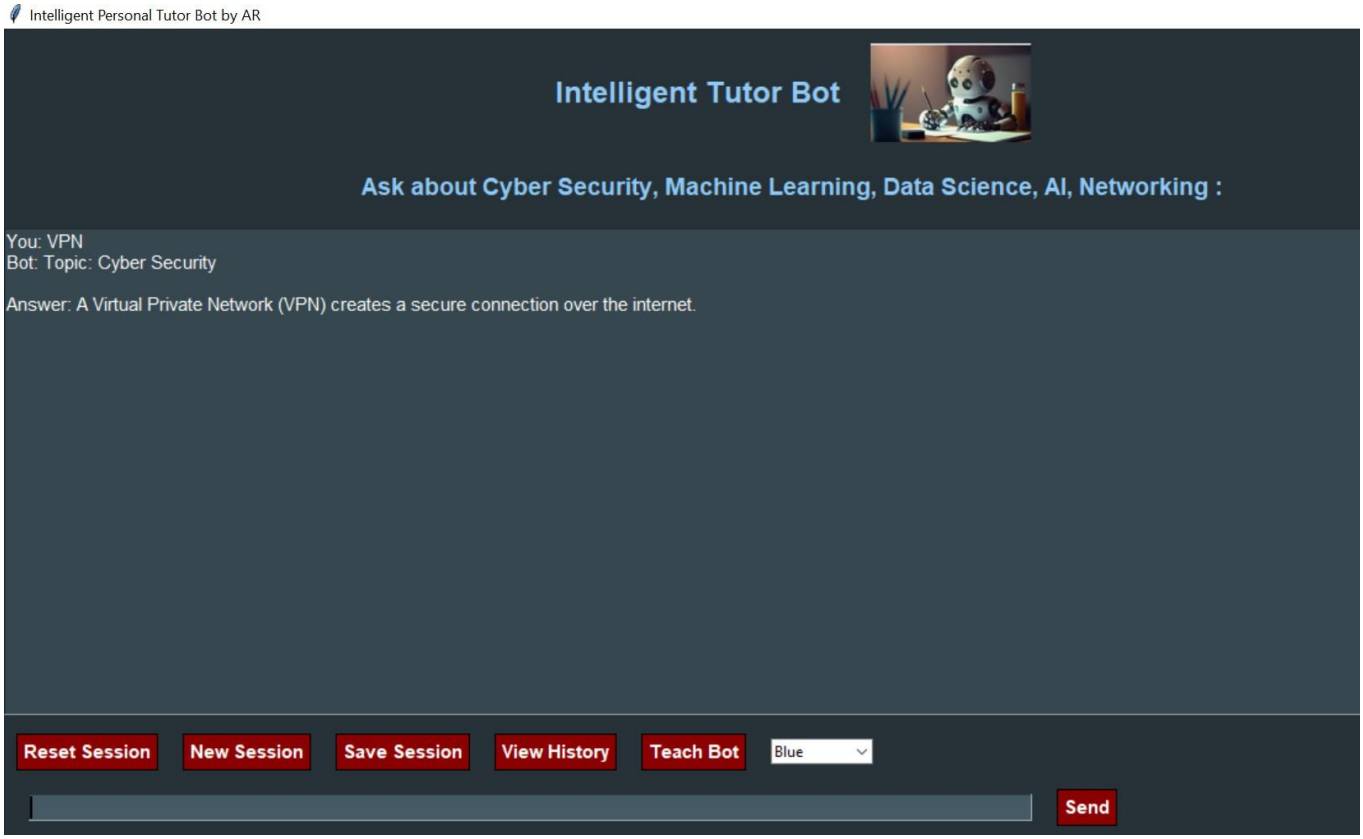
Main interface displaying user query and bot response with Light theme . now change theme to dark


Intelligent Personal Tutor Bot by AR
Intelligent Tutor Bot
Ask about Cyber Security, Machine Learning, Data Science, AI, Networking :
Reset Session
New Session
Save Session
View History
Teach Bot
Dark
Send

Now we change theme from dark to blue .

I have enter a Topic and tab send then answer is display in panel .

Intelligent Personal Tutor Bot by AR
Intelligent Tutor Bot
Ask about Cyber Security, Machine Learning, Data Science, AI, Networking :
Reset Session
New Session
Save Session
View History
Teach Bot
Blue
VPN
Send

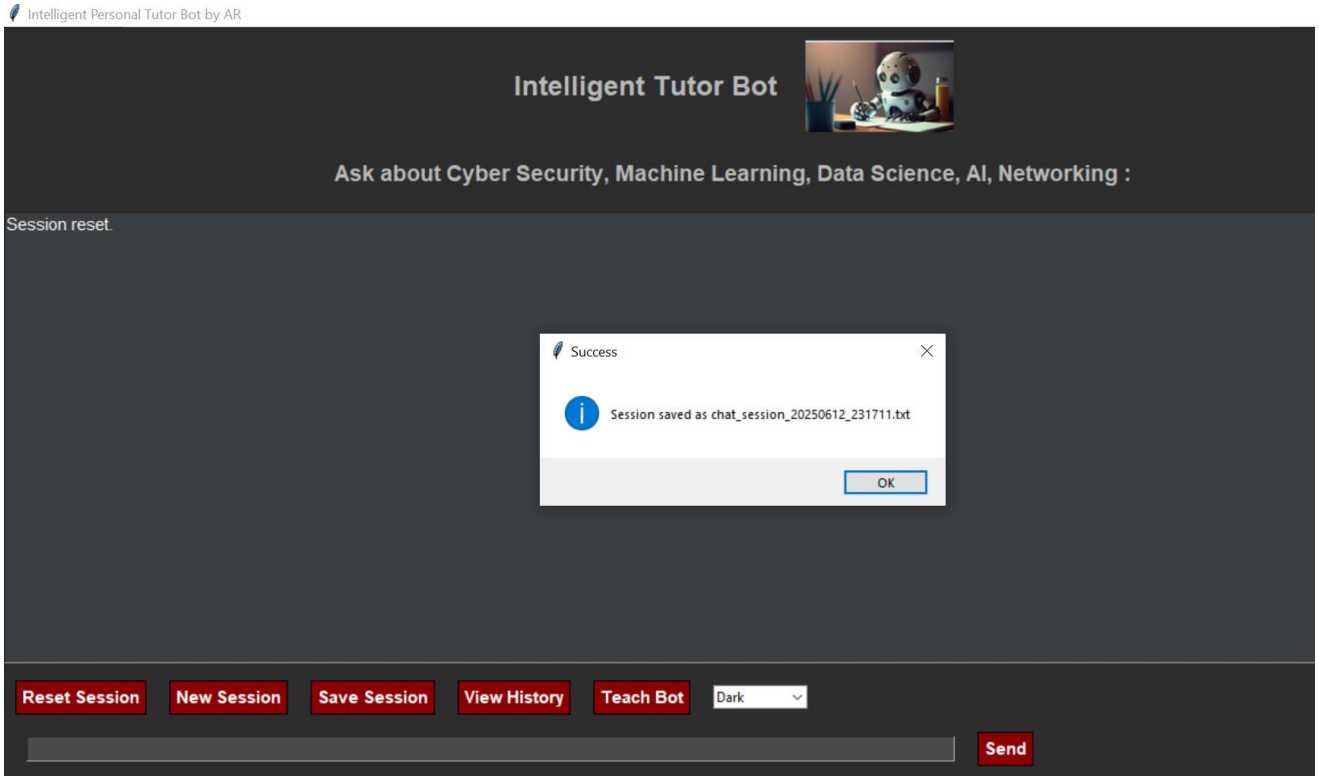


Now Different Topics search and their answer is displayed .

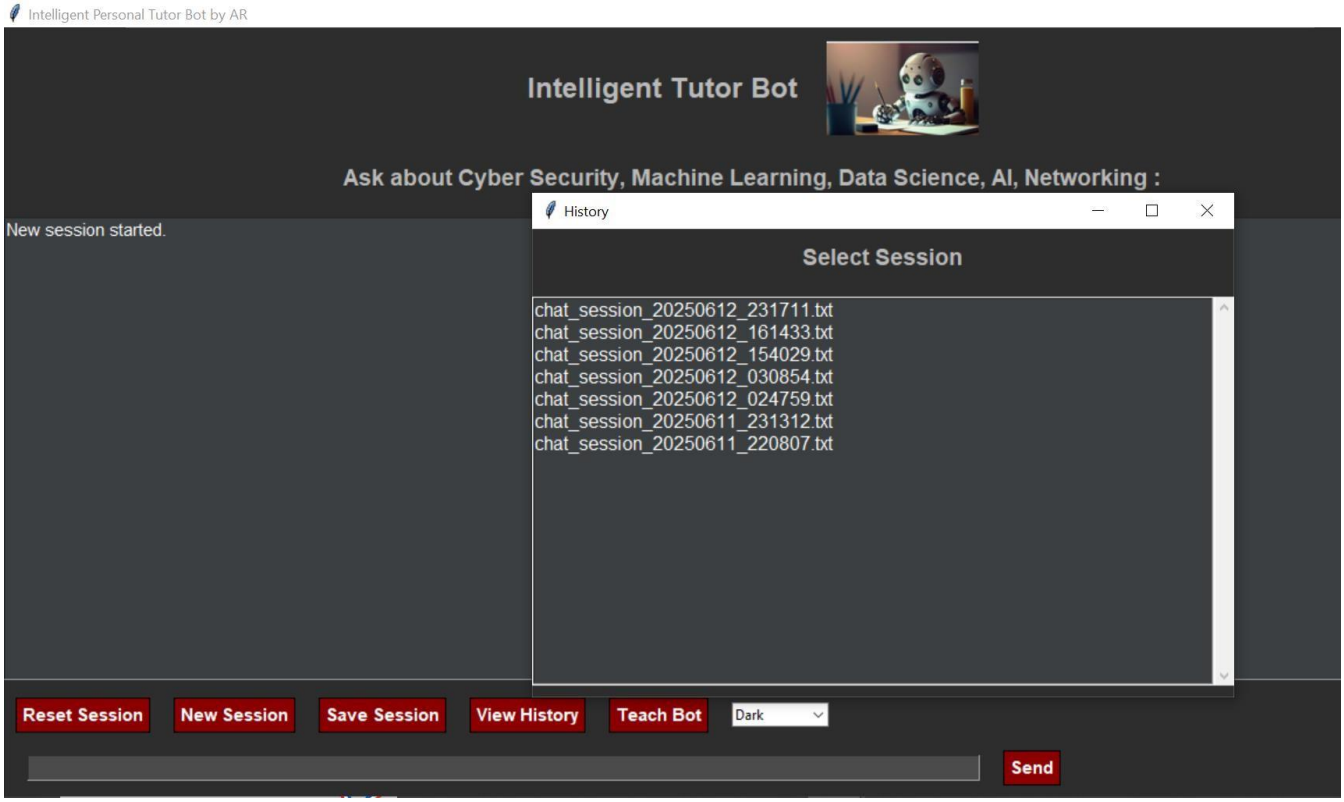
Intelligent Personal Tutor Bot by AR

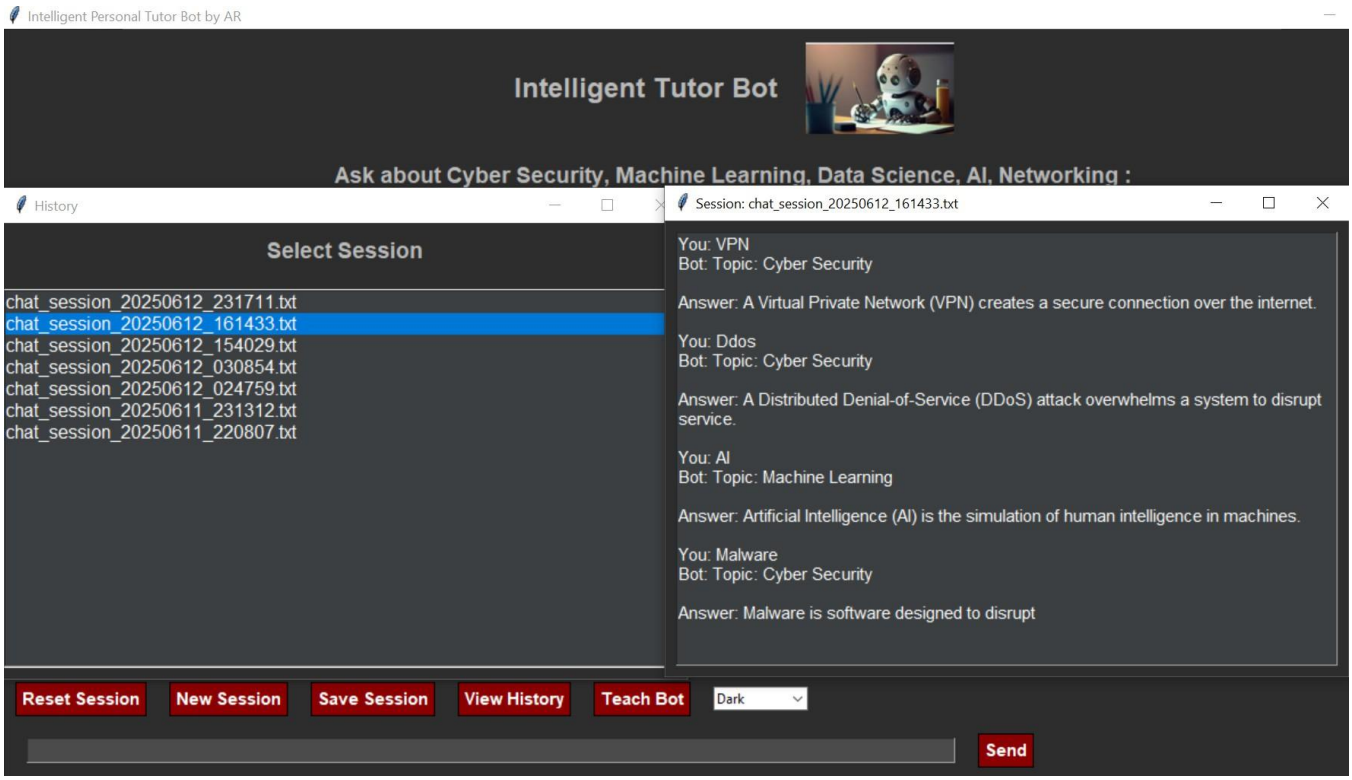
Now Reset Session

New Session


Intelligent Personal Tutor Bot by AR
Intelligent Tutor Bot
Ask about Cyber Security, Machine Learning, Data Science, AI, Networking :
Session reset.
Success
Session saved as chat_session_20250612_231711.txt
OK
Reset Session
New Session
Save Session
View History
Teach Bot
Dark
Send

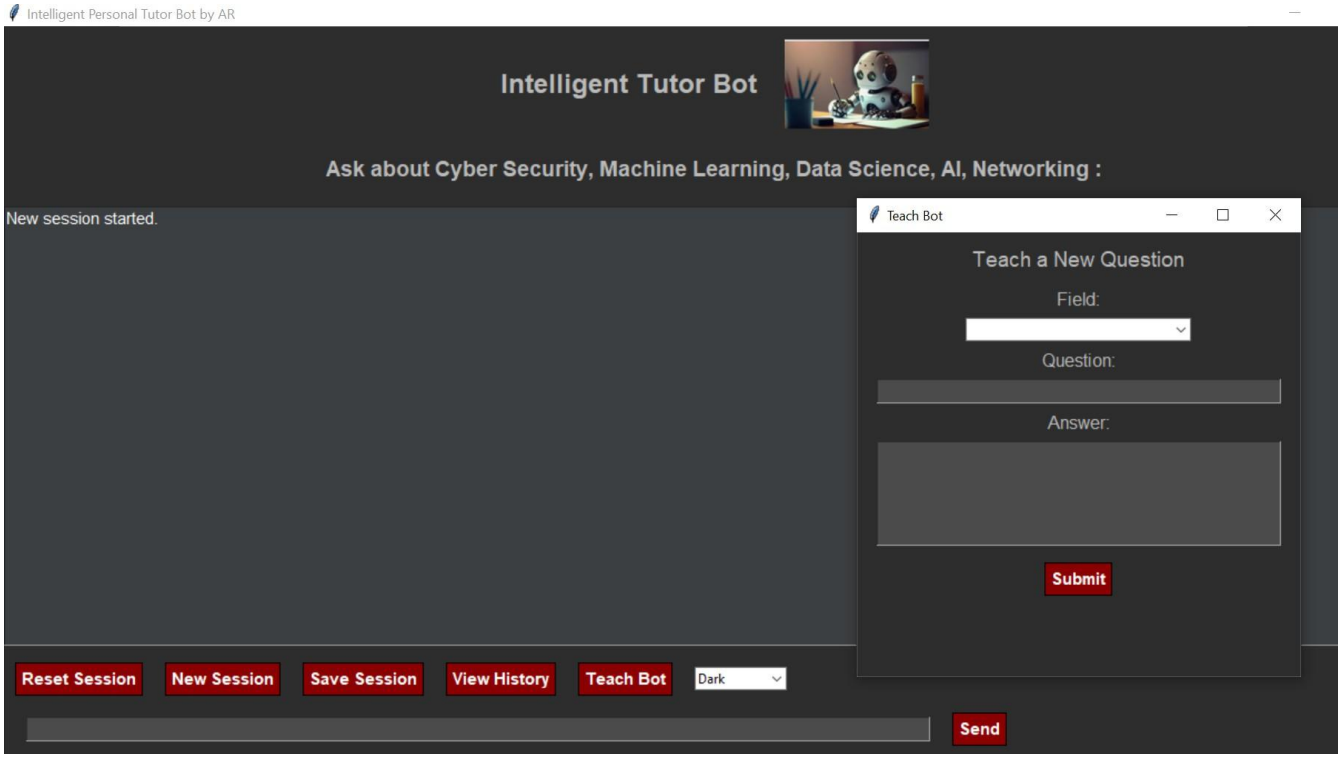
View History it will display all



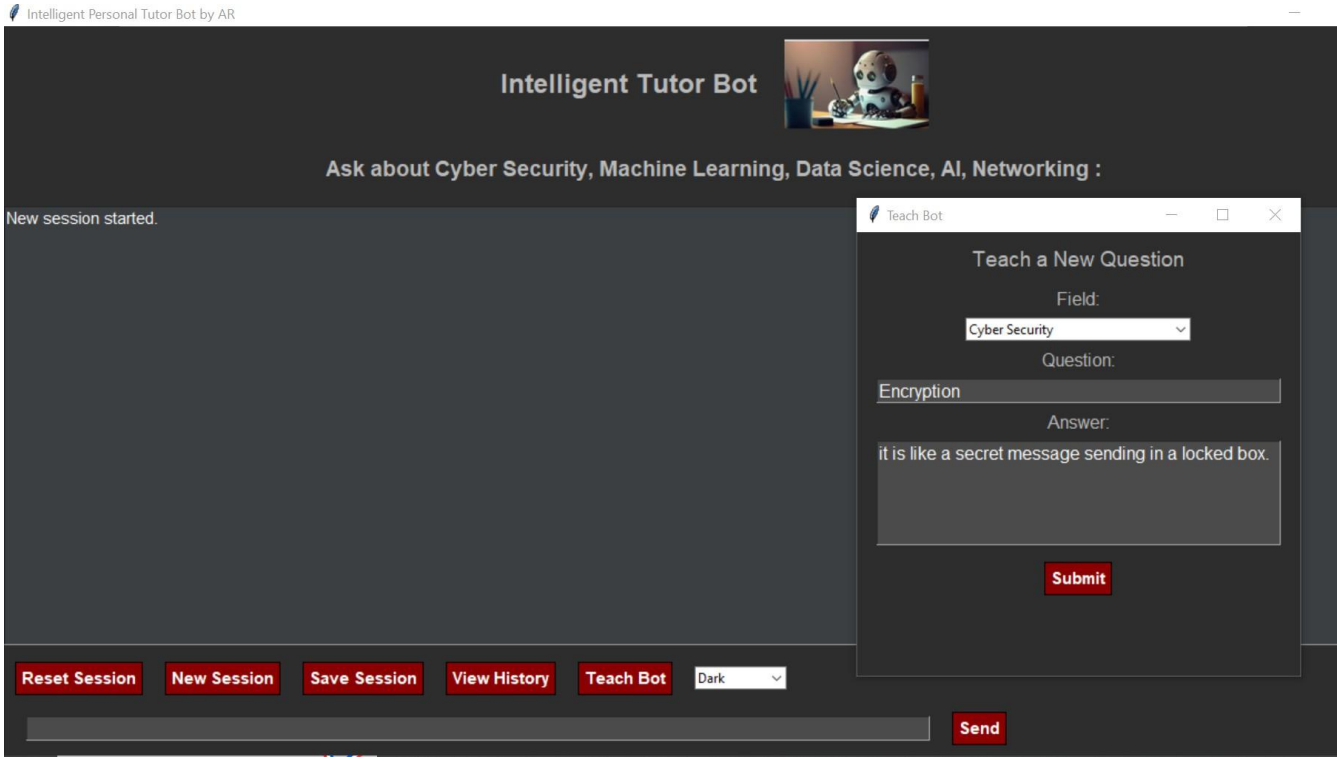


So we can easily view our previous history

When Clicking on Teach Chatbot this Icon Appears .

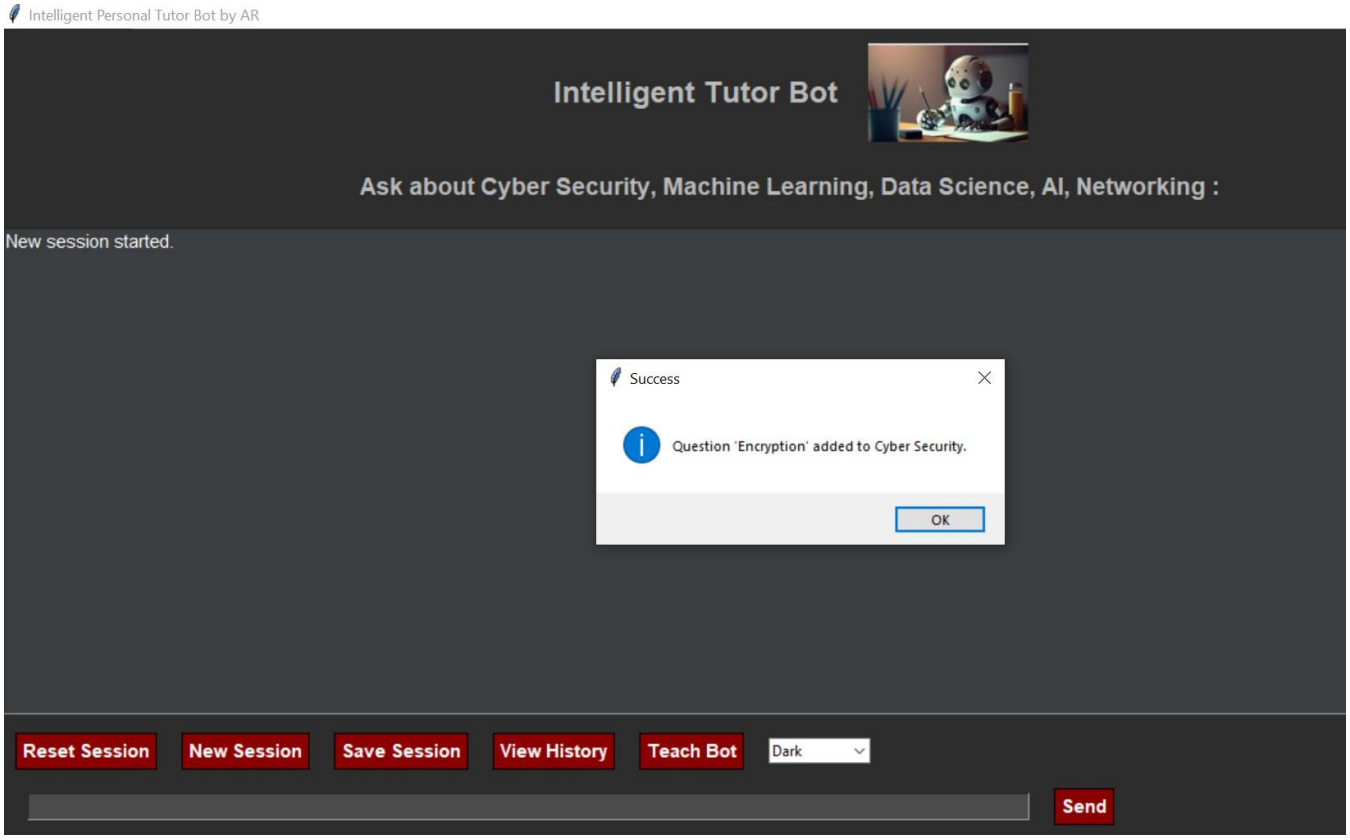


So here I choose field I add question add its answer



And then press submit

The Question is added in Cyber Security Portion



Challenges Faced

- **Knowledge Graph Design:**

  Complex structure and error handling for malformed CSV data.

- **Sentiment Classifier:**

  Low accuracy due to limited training data.

- **Search Optimization:**

  Balancing relevance and weights in Best First Search was tricky.

- **GUI Performance:**

  Ensuring smooth theme switching and responsiveness.

- **Dynamic Updates:**

  Safely syncing GUI, CSV, and graph without concurrency issues.

Limitations

- **Small Sentiment Dataset:**

  Affects nuanced input detection.
- **Static Knowledge Base:**

Limited to predefined CSV entries.

- **Search Depth:**

  May miss deep links in the graph.

- **No Web Integration:**

  Can't fetch answers beyond stored data.

- **Tkinter Scalability:**

  May lag with large or frequent updates.

Future Enhancements

- **Improve Sentiment Analysis:**

  Use larger datasets or transformer models.

- **Add Web Search:**

  Fetch answers beyond local graph.

- **Database Storage:**

  Replace CSV with SQLite for scalability.

- **Voice Input:**

  Add speech-to-text for hands-free use.

- **Multilingual Support:**

  Handle queries in multiple languages.

- **Adaptive Learning:**

  Use reinforcement learning to improve over time.

- **API Integration**

  It will Fetch latest info beyond your static knowledge base.

Conclusion :

The Intelligent Personal Tutor Bot successfully delivers a robust, interactive platform for academic assistance, combining rule-based logic, machine learning, and a semantic network to address student queries. The Tkinter- based GUI enhances user experience with features like theme switching, session management, and auto-suggestions. While the bot effectively handles queries within its

knowledge graph and adapts responses based on sentiment, limitations such as a small sentiment dataset and lack of web integration highlight areas for improvement. Future enhancements, including advanced search algorithms and cloud-based storage, promise to make the bot a more versatile and powerful tool for education. This project demonstrates a strong foundation

---